

Lập trình GIS

Python cơ bản



Phần mềm sử dụng cho môn học



www.anaconda.com



Visual Studio Code

code.visualstudio.com



colab.research.google.com



www.python.org



jupyter.org

Công cụ sử dụng

❖ Dịch vụ online

Google Colab

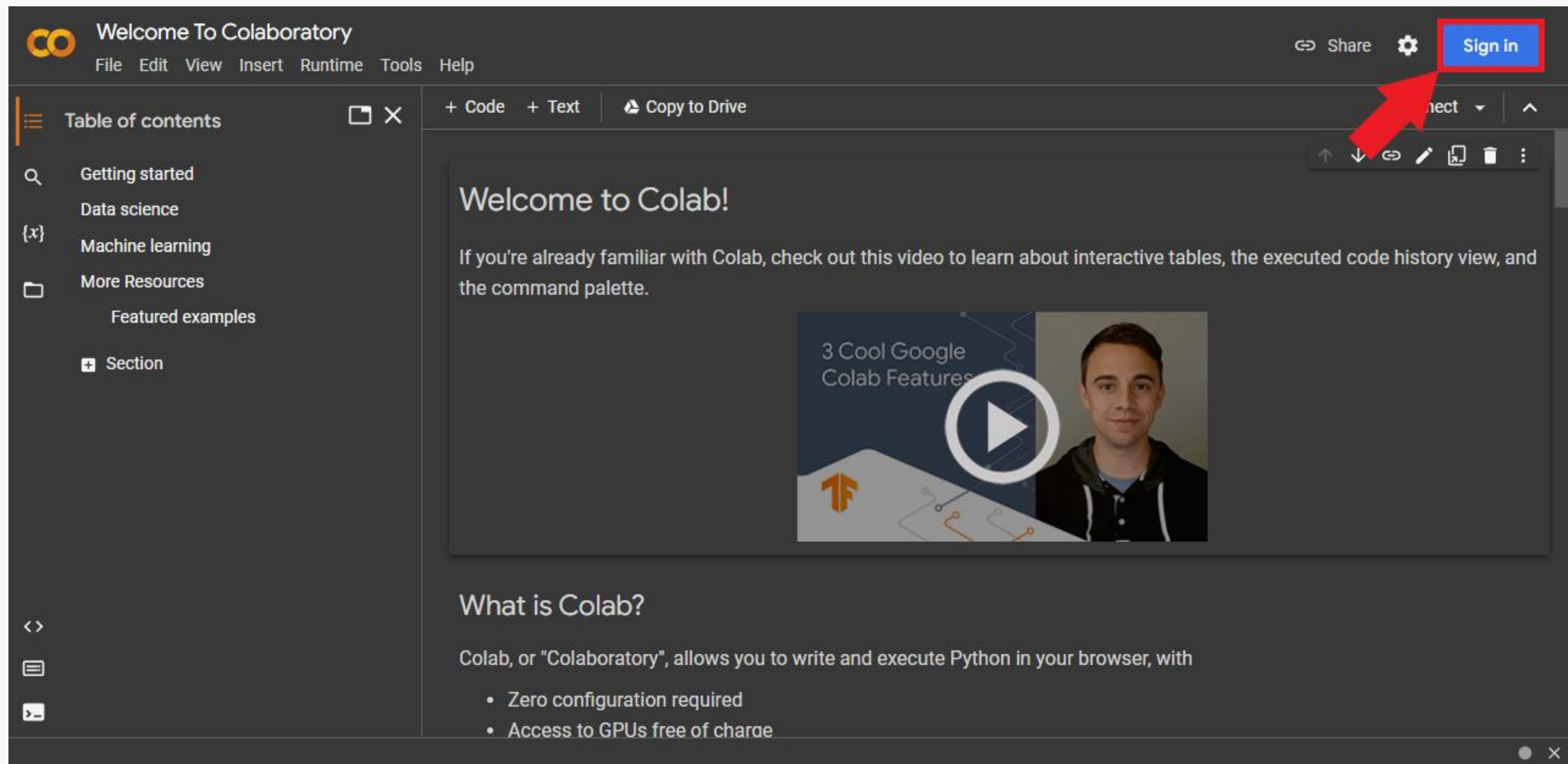
❖ Chạy local

- JupyterLab

- Visual Studio Code

Công cụ sử dụng

Google Colab - colab.research.google.com



Công cụ sử dụng

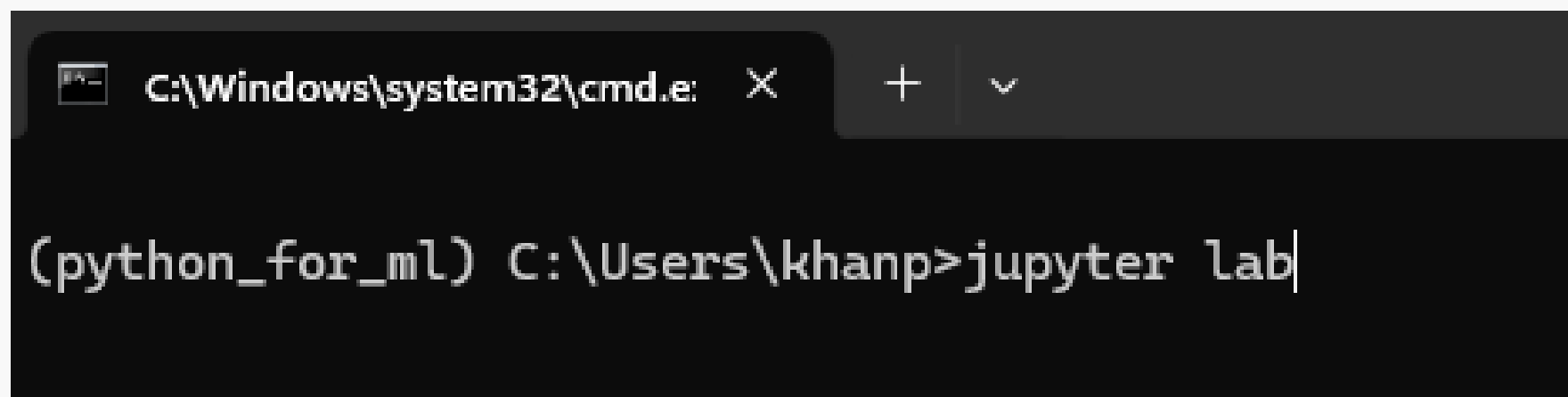
JupyterLab

- Để cài đặt JupyterLab ta chạy lệnh sau:

`pip install jupyterlab`

- Để chạy thì ta dùng lệnh sau:

`jupyter lab`



```
C:\Windows\system32\cmd.e: X + v  
(python_for_ml) C:\Users\khanp>jupyter lab
```

To access the server, open this file in a browser:

`file:///C:/Users/khanp/AppData/Roaming/jupyter/runtime/jpserver-14596-open.html`

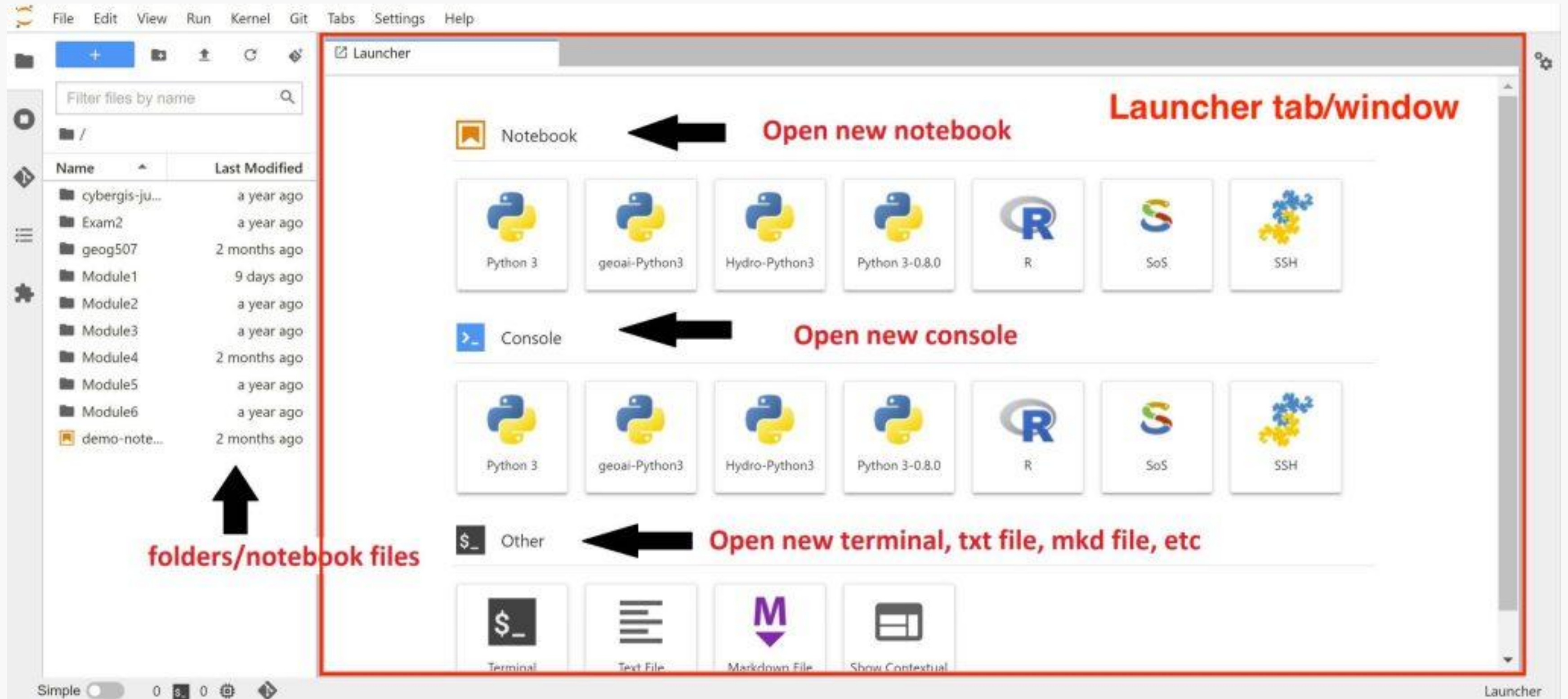
Or copy and paste one of these URLs:

`http://localhost:8888/lab?token=c0a260c64c46fb0bfa52607d6095ffa508cdd91eff190b9`

`http://127.0.0.1:8888/lab?token=c0a260c64c46fb0bfa52607d6095ffa508cdd91eff190b9`

Công cụ sử dụng

JupyterLab



Python cơ bản

Biến và kiểu dữ liệu

Python cơ bản

Biến và kiểu dữ liệu

- ❖ *Định nghĩa và sử dụng biến trong Python*
- ❖ *Nhận diện và sử dụng các kiểu dữ liệu khác nhau trong Python, bao gồm số nguyên, số thực, chuỗi, boolean, list và dictionary.*
- ❖ *Hiểu cách các kiểu dữ liệu được sử dụng trong bối cảnh dữ liệu địa lý, chẳng hạn như tọa độ và thông tin thuộc tính.*
- ❖ *Thực hiện các thao tác và thao tác cơ bản trên các kiểu dữ liệu khác nhau.*
- ❖ *Áp dụng được các khái niệm về biến và kiểu dữ liệu vào các vấn đề địa lý thực tế như tính toán tâm và quản lý các thuộc tính của dữ liệu.*

Python cơ bản

Biến và kiểu dữ liệu

❖ Trong Python, biến là tên tượng trưng dùng để tham chiếu hoặc trỏ đến một đối tượng. Khi một đối tượng được gán cho một biến, bạn có thể tham chiếu đến đối tượng đó bằng tên biến.

❖ Tạo ra 1 biến đơn giản như sau:

```
num_points = 150
```

❖ Để xem giá trị của biến ta dùng lệnh sau:

```
print(num_points)
```

150

Python cơ bản

Biến và kiểu dữ liệu

Đặt tên biến - Khi đặt tên biến ta sẽ có những quy tắc như sau:

- ❖ Tên biến phải bắt đầu bằng một chữ cái hoặc dấu gạch dưới, chẳng hạn như `_`
- ❖ Phần còn lại của tên biến có thể bao gồm các chữ cái, số và dấu gạch dưới.
- ❖ Tên biến phân biệt chữ hoa chữ thường, do đó `num_points` và `Num_Points` là các biến khác nhau.
- ❖ Tên biến phải mang tính mô tả và có ý nghĩa như `num_points` thay vì `n`.

Python cơ bản

Biến và kiểu dữ liệu

❖ Tránh sử dụng các từ khóa Python và các hàm dựng sẵn làm tên biến, chẳng hạn như:

print, sum, list, dict, str, int, float, bool, set, tuple, range, type, object, None, True, False, and, or, not, if, else, elif, for, while, break, continue, pass, def, return, lambda, class, import, from, as, with, try, except, finally, raise, assert, del, in, is, global, nonlocal, yield, async, await.

Python cơ bản

Biến và kiểu dữ liệu

Python hỗ trợ nhiều kiểu dữ liệu khác nhau, điều này rất cần thiết để hiểu trước khi làm việc với dữ liệu không gian địa lý. Các kiểu dữ liệu phổ biến nhất bao gồm:

❖ **Integers (int):** Đây là các số nguyên, ví dụ: 1, 150, -5

num_features = 500 # Khai báo số lượng các tính năng trong một tập dữ liệu không gian địa lý

❖ **Số thập phân (float):** Đây là những số có dấu thập phân, ví dụ: 3.14, -0.001, 100.0

latitude = 35.6895 # Khai báo vĩ độ của một điểm trên bề mặt Trái Đất

longitude = 139.6917 # Khai báo kinh độ của một điểm trên bề mặt Trái Đất

Python cơ bản

Biến và kiểu dữ liệu

❖ **Kiểu chuỗi (str):** Ví dụ “Xin chào”, “Du lieu khong gian”, “Lat/Long”

coordinate_system = "WGS 84" # Khai báo hệ tọa độ thường dùng

Chuỗi có thể được đặt trong dấu ngoặc đơn (') hoặc dấu ngoặc kép (").

Bạn cũng có thể sử dụng dấu ngoặc kép (''' hoặc ''') cho chuỗi nhiều dòng

❖ **Boolean (bool):** Boolean thể hiện một trong hai giá trị: True hoặc False

is_geo = True

❖ **List:** Danh sách là tập hợp các mục được sắp xếp theo thứ tự, có thể thuộc bất kỳ kiểu dữ liệu nào.

coordinates = [

32.6835,

129.6612] # Một danh sách thể hiện vĩ độ và kinh độ của một điểm

Python cơ bản

Biến và kiểu dữ liệu

- ❖ Kiểu từ điển (dict): Từ điển là tập hợp các cặp **key-value**

```
feature_attributes = {  
    "name": "Mount Fuji",  
    "height_meters": 3776,  
    "type": "Stratovolcano",  
    "location": [35.3606, 138.7274],  
}
```

Python cơ bản

Ký tự đặc biệt

- ❖ Ta có thể sử dụng ký tự **`\n`** để xuống dòng mới

Ví dụ: `print("Hello World!\nThis is a Python script.")`

Kết quả:

Hello World!

This is a Python script.

- ❖ Khi muốn chèn 1 tab, ta có thể sử dụng **`\t`**

Ví dụ: `print("This is the first line.\n\tThis is the second line. It is indented.")`

Kết quả:

This is the first line.

This is the second line. It is indented.

Python cơ bản

Ký tự đặc biệt

❖ Chú thích ta dùng ký tự #

Đây là chú thích

num_points = 120 # Đây là chú thích trên từng dòng code

Python cơ bản

Thao tác giữa biến và kiểu dữ liệu

Thực hiện một số thao tác cơ bản với các biến

❖ Cộng thêm giá trị vào biến `num_features`

```
num_features += 20
```

```
print("Giá trị biến số đặc trưng đã được cập nhật:", num_features)
```

Kết quả:

```
Giá trị biến số đặc trưng đã được cập nhật: 520
```

Python cơ bản

Thao tác giữa biến và kiểu dữ liệu

- ❖ Chuyển đổi vĩ độ từ độ sang radian (cần thiết cho một số tính toán không gian địa lý):

```
import math
```

```
latitude = 35.6895
```

```
latitude_radians = math.radians(latitude)
```

```
print("Vĩ độ theo đơn vị radian:", latitude_radians)
```

Kết quả

```
Vĩ độ theo đơn vị radian: 0.6228992833905163
```

Python cơ bản

Thao tác giữa biến và kiểu dữ liệu

❖ Thêm các tọa độ mới vào một danh sách có sẵn

```
coordinates = [35.6895, 139.6917] #danh sách có sẵn
```

```
coordinates.append(34.0522) # Thêm vĩ độ mới
```

```
coordinates.append(-118.2437) # Thêm kinh độ mới
```

```
print("DS tọa độ đã cập nhật:", coordinates)
```

Kết quả:

```
DS tọa độ đã cập nhật: [35.6895, 139.6917, 34.0522, -118.2437]
```

Python cơ bản

Thao tác giữa biến và kiểu dữ liệu

❖ Truy xuất 1 thành phần trong biến dictionary

```
mount_fuji_name = feature_attributes["name"]
```

```
mount_fuji_height = feature_attributes["height_meters"]
```

```
print(f"{mount_fuji_name} là {mount_fuji_height} mét")
```

Kết quả:

```
Mount Fuji là 3776 mét
```


Python cơ bản

Ví dụ

Ta có một danh sách điểm tọa độ gồm có vĩ độ (latitude) và kinh độ (longitude) tính tâm (điểm trung bình)

```
points = [  
    [35.6895, 139.6917],  
    [34.0522, -118.2437],  
    [51.5074, -0.1278],  
    [48.8566, 2.3522],  
]
```

Python cơ bản

Ví dụ

Tính tâm:

```
centroid_lat = sum([point[0] for point in points]) / len(points)  
centroid_lon = sum([point[1] for point in points]) / len(points)  
centroid = [centroid_lat, centroid_lon]  
print("Tâm của các điểm là:", centroid)
```

Kết quả:

```
Tâm của các điểm là: [42.526425, 5.91809999999999998]
```

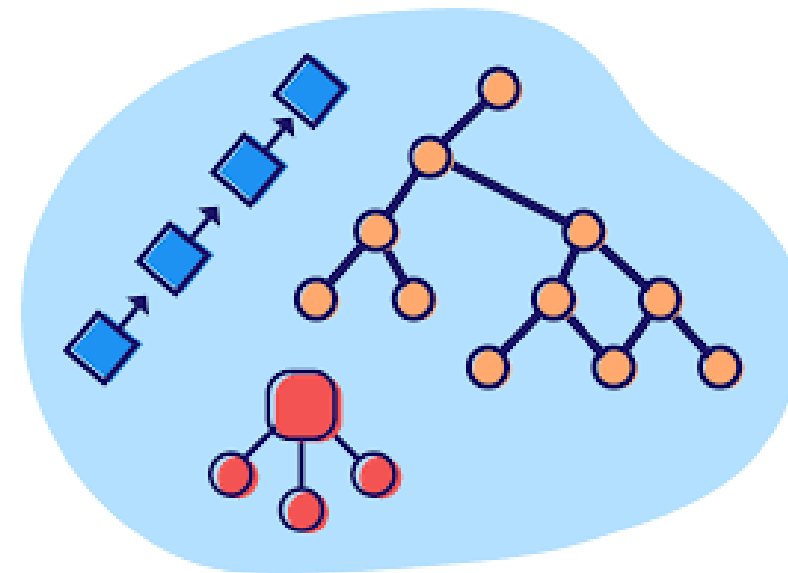

Python cơ bản

Bài tập 1

1. Tạo danh sách các bộ dữ liệu, mỗi bộ dữ liệu đại diện cho tọa độ (vĩ độ, kinh độ) của các thành phố khác nhau mà bạn đã ghé thăm.
2. Tính toán tâm của các tọa độ này.
3. Tạo một từ điển để lưu trữ vĩ độ và kinh độ của tâm.

Python cơ bản

Cấu trúc dữ liệu



Python cơ bản

Cấu trúc dữ liệu

- ❖ *Hiểu được các đặc điểm và trường hợp sử dụng của tuple, list, set, dictionary*
- ❖ *Các cấu trúc dữ liệu này để lưu trữ và thao tác dữ liệu không gian địa lý, chẳng hạn như tọa độ, đường dẫn (path) và thông tin thuộc tính.*
- ❖ *Phân biệt giữa các cấu trúc dữ liệu có thể thay đổi và không thể thay đổi và chọn cấu trúc phù hợp cho các tác vụ không gian địa lý khác nhau.*
- ❖ *Thực hiện các thao tác chung trên các cấu trúc dữ liệu này, bao gồm lập chỉ mục, cắt lát, thêm/xóa phần tử và cập nhật giá trị.*
- ❖ *Sử dụng từ điển để quản lý các thuộc tính đối tượng không gian địa lý và hiểu được tầm quan trọng của các cặp khóa-giá trị trong quản lý dữ liệu không gian địa lý.*

Python cơ bản

Cấu trúc dữ liệu

❖ Tuple

Tuple là chuỗi bất biến, nghĩa là sau khi tuple được tạo, các phần tử của nó không thể thay đổi.

Ví dụ, tuple có thể được sử dụng để lưu trữ tọa độ của một điểm địa lý (vĩ độ, kinh độ).

```
point = (  
    35.6895,  
    139.6917,  
)
```

Python cơ bản

Cấu trúc dữ liệu

❖ Tuple

Truy xuất các thành phần trong một tuple bằng cách sử dụng chỉ mục:

```
latitude = point[0]
```

```
longitude = point[1]
```

```
print(f"Latitude: {latitude}, Longitude: {longitude}")
```

Kết quả:

```
Latitude: 35.6895, Longitude: 139.6917
```


Python cơ bản

Cấu trúc dữ liệu

❖ List

- List là các chuỗi có thứ tự, có thể thay đổi.
- Ta có thể thay đổi, thêm hoặc xóa các phần tử sau khi danh sách đã được tạo.
- Danh sách rất linh hoạt và có thể lưu trữ nhiều loại dữ liệu
- Ví dụ: Ta có thể lưu trữ danh sách tọa độ biểu diễn đường dẫn hoặc ranh giới.

```
path = [  
    (35.6895, 139.6917),  
    (34.0522, -118.2437),  
    (51.5074, -0.1278),  
]
```

Python cơ bản

Cấu trúc dữ liệu

❖ *List*

- *Ta có thể thêm một vị trí mới vào biến path*

path.append((48.8566, 2.3522)) # Adding Paris to the path

print("Path đã được cập nhật:", path)

Kết quả:

*Path đã được cập nhật: [(35.6895, 139.6917), (34.0522, -118.2437),
(51.5074, -0.1278), (48.8566, 2.3522)]*

Python cơ bản

Cấu trúc dữ liệu

❖ *List (tiếp theo)*

List cho phép ta thực hiện nhiều thao tác khác nhau như slicing, cho phép ta truy cập vào một tập hợp con của list.

```
sub_path = path[:2] # cắt 1 vị trí đầu tiên của biến path
```

```
print("Sub-path:", sub_path)
```

Kết quả:

```
Sub-path: [(35.6895, 139.6917), (34.0522, -118.2437)]
```


Python cơ bản

Cấu trúc dữ liệu

❖ Set

Set là tập hợp các phần tử duy nhất không theo thứ tự, không lưu các giá trị trùng nhau

```
regions = ["North America", "Europe", "Asia"] # Set of regions
```

```
regions = set(regions)
```

- Thêm một khu vực mới

```
regions.add("Africa")
```

```
print("Khu vực được cập nhật:", regions)
```

Kết quả:

```
Khu vực được cập nhật: {'Africa', 'North America', 'Asia', 'Europe'}
```

Python cơ bản

Cấu trúc dữ liệu

❖ *Set (tiếp theo)*

Thêm dữ liệu bị trùng đang tồn tại thì tập dữ liệu vẫn không thay đổi

```
regions.add("Europe")
```

```
print("Ket qua sau khi them du lieu trung:", regions)
```

Kết quả

```
Ket qua sau khi them du lieu trung: {'Africa', 'North America', 'Asia',  
'Europe'}
```

Python cơ bản

Cấu trúc dữ liệu

❖ Dictionary

- Dict là tập hợp các cặp khóa-giá trị, trong đó mỗi khóa là duy nhất.
- Từ điển cực kỳ hữu ích để lưu trữ dữ liệu được liên kết với các mã định danh cụ thể, chẳng hạn như dữ liệu thuộc tính của các đối tượng địa lý.

Ví dụ: bạn có thể sử dụng từ điển để lưu trữ các thuộc tính của một đối tượng không gian địa lý, chẳng hạn như thông tin của một thành phố Tokyo.

```
city_attributes = {  
    "name": "Tokyo",  
    "population": 13929286,  
    "coordinates": (35.6895, 139.6917),  
}
```

Python cơ bản

Cấu trúc dữ liệu

❖ *Dictionary (tiếp theo)*

- *Ta có thể truy xuất dữ liệu Dict bằng key*

```
city_name = city_attributes["name"]
```

```
city_population = city_attributes["population"]
```

```
print(f"City: {city_name}, Population: {city_population}")
```

Kết quả:

```
City: Tokyo, Population: 13929286
```

Python cơ bản

Cấu trúc dữ liệu

❖ *Dictionary (tiếp theo)*

- *Cập nhật giá trị Dict bằng key*

```
city_attributes["area_km2"] = 2191 # Thêm diện tích của thành phố Km2
```

```
print("Thuộc tính của thành phố đã được cập nhật:", city_attributes)
```

Kết quả

```
Thuộc tính của thành phố đã được cập nhật:{'name': 'Tokyo',  
'population': 13929286, 'coordinates': (35.6895, 139.6917), 'area_km2': 2191}
```


Python cơ bản

Cấu trúc dữ liệu

Bài tập cấu trúc dữ liệu:

Tạo một dict để lưu trữ các thuộc tính dữ liệu địa lý của các ngọn núi thuộc tỉnh Bà Rịa – Vũng Tàu, sau đó thêm thuộc tính chiều cao của ngọn núi đã có trong dict và hiển thị tất cả các thuộc tính có trong dict đó.

Python cơ bản

Thao tác với chuỗi

Python cơ bản

Thao tác với chuỗi

- ❖ *Tạo và thao tác chuỗi trong Python bao gồm nối và lặp lại.*
- ❖ *Các phương thức chuỗi như `lower()`, `upper()`, `strip()`, `replace()` và `split()` để xử lý dữ liệu không gian địa lý.*
- ❖ *Định dạng chuỗi bằng phương thức `format()` và `f-strings` để đưa dữ liệu biến vào chuỗi.*
- ❖ *Phân tích cú pháp và trích xuất thông tin cụ thể từ chuỗi, chẳng hạn như tọa độ hoặc tên vị trí.*
- ❖ *Sử dụng các thao tác chuỗi trong các tác vụ không gian địa lý thực tế, nâng cao khả năng làm việc và quản lý dữ liệu địa lý của mình.*

Python cơ bản

Thao tác với chuỗi

❖ *Tạo và thao tác chuỗi*

- *Chuỗi trong Python là chuỗi ký tự. Bạn có thể tạo chuỗi bằng cách đặt các ký tự trong dấu ngoặc đơn hoặc dấu ngoặc kép.*

location_name = "Bà Rịa"

- *Ta có thể nối các chuỗi bằng toán tử +:*

location_name_full = location_name + " – Vũng Tàu"

print(location_name_full)

Kết quả

Bà Rịa – Vũng Tàu

Python cơ bản

Thao tác với chuỗi

❖ *Tạo và thao tác chuỗi (tiếp theo)*

- *Bạn cũng có thể lặp lại chuỗi bằng toán tử *:*

```
separator = "-" * 10
```

```
print(separator)
```

Kết quả

```
-----
```

Python cơ bản

Thao tác với chuỗi

❖ *Tạo và thao tác chuỗi (tiếp theo)*

- *Xây dựng một chuỗi truy vấn SQL cơ bản để chọn dữ liệu từ cơ sở dữ liệu không gian địa lý:*

```
table_name = "locations"
```

```
condition = "province = 'Lam Dong'"
```

```
sql_query = f"SELECT * FROM {table_name} WHERE {condition};"
```

```
print(sql_query)
```

Kết quả

```
SELECT * FROM locations WHERE province = ' Lam Dong ';
```

Python cơ bản › Thao tác với chuỗi

❖ Các phương thức chuỗi dùng cho dữ liệu không gian

Python cung cấp nhiều phương thức tích hợp để thao tác chuỗi. Một số phương thức thường dùng bao gồm:

- ***lower(), upper()***: Chuyển đổi chuỗi thành chữ thường hoặc chữ hoa.
- ***strip()***: Xóa khoảng trắng đầu và cuối.
- ***lstrip(), rstrip()***: Xóa khoảng trắng đầu và cuối.
- ***replace()***: Thay thế chuỗi con bằng chuỗi con khác.
- ***split()***: Chia chuỗi thành danh sách các chuỗi con dựa trên dấu phân cách.
- ***join()***: Nối danh sách các chuỗi thành một chuỗi duy nhất bằng dấu phân cách được chỉ định.

Python cơ bản › Thao tác với chuỗi

❖ Các phương thức chuỗi dùng cho dữ liệu không gian

- ***upper()***

```
location_name_upper = location_name.upper()
```

```
print(" Vị trí đã đổi thành chữ hoa: {location_name_upper}")
```

Kết quả

```
Vị trí đã đổi thành chữ hoa: BÀ Rịa
```

- ***strip()***

```
location_name_clean = location_name.strip()
```

```
print(location_name_clean)
```

Kết quả

```
Bà Rịa
```


Python cơ bản › Thao tác với chuỗi

❖ Các phương thức chuỗi dùng cho dữ liệu không gian

- ***replace()***

```
location_name_replaced = location_name.replace("Rịa", "Đen")  
print(location_name_replaced) # Thay thế 'Rịa' với 'Đen'
```

Kết quả

Bà Đen

- ***split()***

```
location_parts = location_name_full.split(" - ")  
print(location_parts)
```

Kết quả

['Bà Đen', 'Vũng Tàu']

Python cơ bản › Thao tác với chuỗi

❖ Các phương thức chuỗi dùng cho dữ liệu không gian

Giả sử ta có danh sách tên quốc gia có định dạng không nhất quán và bạn muốn chuẩn hóa chúng:

```
countries = [" vietNam", "JAPAN ", "china ", "Laos"]  
normalized_countries = [country.strip().title() for country in countries]  
print(normalized_countries)
```

Kết quả

```
['Vietnam', 'Japan', 'China', 'Laos']
```

Python cơ bản › Thao tác với chuỗi

❖ Định dạng chuỗi

Định dạng chuỗi là điều cần thiết khi chuẩn bị dữ liệu để xuất hoặc khi bạn cần đưa các giá trị biến vào chuỗi. Bạn có thể sử dụng phương thức format() hoặc f-strings (trong Python 3.6 trở lên) để định dạng chuỗi.

```
latitude = 27.9881
```

```
longitude = 86.9250
```

```
formatted_coordinates = "Coordinates: ({} , {})".format(latitude, longitude)
```

```
print(formatted_coordinates)
```

Kết quả

```
Coordinates: (27.9881, 86.925)
```


Python cơ bản › Thao tác với chuỗi

❖ Định dạng chuỗi (tiếp theo)

```
formatted_coordinates_fstring = f"Coordinates: ({latitude},  
{longitude})"
```

```
print(formatted_coordinates_fstring)
```

Kết quả

```
Coordinates: (27.9881, 86.925)
```

Python cơ bản › Thao tác với chuỗi

❖ *Phân tích và trích xuất thông tin từ chuỗi*

Thông thường, bạn sẽ cần trích xuất thông tin cụ thể từ chuỗi, đặc biệt là khi xử lý dữ liệu địa lý. Ví dụ, bạn có thể cần trích xuất tọa độ từ chuỗi đã định dạng.

```
coordinate_string = "27.9881N, 86.9250E"
```

```
lat_str, lon_str = coordinate_string.split(", ")
```

```
latitude = float(lat_str[:-1]) # Chuyển string thành float và xóa 'N'
```

```
longitude = float(lon_str[:-1]) # Chuyển string thành float và xóa 'E'
```

```
print(f"Parsed coordinates: ({latitude}, {longitude}))"
```

Kết quả

```
Parsed coordinates: (27.9881, 86.925)
```

Python cơ bản › Thao tác với chuỗi

❖ *Phân tích và trích xuất thông tin từ chuỗi (tiếp theo)*

Well-Known Text (WKT) là ngôn ngữ đánh dấu văn bản để biểu diễn các đối tượng hình học vector. Hãy định dạng một chuỗi để biểu diễn hình học POINT:

```
wkt_point = f"POINT({longitude} {latitude})"
```

```
print(wkt_point)
```

Kết quả

```
POINT(86.925 27.9881)
```

Python cơ bản › Thao tác với chuỗi

❖ *Phân tích và trích xuất thông tin từ chuỗi (tiếp theo)*

Ví dụ khác, ta cần truy xuất thông tin đường xa, quận và tỉnh thành từ chuỗi.

```
address = "236B Lê Văn Sỹ, Tân Bình, TP.HCM"
```

```
street, district, city = address.split(", ")
```

```
print(f"Đường: {street}, Quận: {district}, TP: {city}")
```

Kết quả

```
Đường: 236B Lê Văn Sỹ, Quận: Tân Bình, TP: TP.HCM
```

Python cơ bản › Thao tác với chuỗi

Bài tập về chuỗi

1. Tạo một chuỗi biểu diễn tên của một thành phố. Chuyển chuỗi thành chữ thường rồi thành chữ hoa.
2. Lấy một chuỗi có định dạng 'vĩ độ, kinh độ' (ví dụ: '11.162235N, 106.625061E') và trích xuất các giá trị số của vĩ độ và kinh độ.
3. Tạo một chuỗi được định dạng bao gồm tên của một vị trí và tọa độ của vị trí đó. Sử dụng cả phương thức `format()` và `f-strings` để thực hiện việc này.
4. Thay thế một chuỗi con trong tên của một địa điểm (ví dụ: đổi 'Bình Phước' thành 'Bình Dương') và in kết quả.
5. Cho một danh sách các địa chỉ theo định dạng "Đường, Quận, Tỉnh thành", hãy viết một hàm để phân tích cú pháp và trả về một từ điển có các khóa là đường, quận và thành phố.
6. Viết một hàm chuyển đổi một cặp tọa độ vĩ độ và kinh độ thành chuỗi WKT POINT.

Thank
You!